UNISENAI

Docker Avançado

Redes, Dockerfile e Publicação

Prof. Nisflei Galoni

DevOps - IaC

Containers

Networks

Dockerfile

Registry

Agenda da Aula

1

Docker Networking

Conceitos de redes, drivers e isolamento

2

Port Mapping

Mapeamento de portas e exposição de serviços

3

Dockerfile Basics

Estrutura e instruções principais

4

Building Images

Comando docker build e otimizações

5

Docker Hub Publishing

Publicação e versionamento de imagens

6

Atividade Prática

Hands-on com todos os conceitos

Docker Networking: Conceitos Fundamentais

O que é Docker Networking?

Sistema que permite **comunicação entre containers**, entre containers e o host, e acesso a redes externas.

Isolamento

Cada container possui seu próprio **network namespace**, garantindo isolamento de rede e segurança.

Drivers

Docker suporta múltiplos **drivers de rede** para diferentes casos de uso e arquiteturas.

Bridge

Driver padrão. Conecta containers em uma rede virtual isolada.

Uso: Aplicações single-host

Host

Container compartilha a stack de rede do host. Sem isolamento.

Uso: Performance máxima

Overlay

Conecta múltiplos hosts Docker. Usado em Swarm/Kubernetes.

Uso: Multi-host, orquestração

None

Sem rede. Container completamente isolado.

Uso: Isolamento total

Bridge Network & DNS

O Switch Virtual

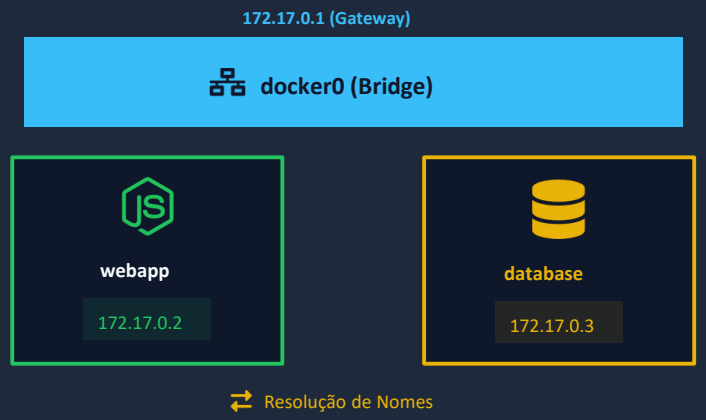
A rede **bridge** atua como um switch de software. Containers conectados a ela recebem um IP interno e podem se comunicar livremente.

DNS Mágico

O Docker mantém um servidor DNS interno. Você não precisa saber o IP do banco de dados, basta usar o **nome do container**.

Dentro do container "webapp"

```
root@webapp:/# ping database
PING database (172.17.0.3): 56 data bytes
 64 bytes from 172.17.0.3: icmp_seq=0 ttl=64 time=0.05 ms
```




```
# Criar a rede (se ainda não existir)
oot@webapp:/#  docker network create bigdata-net

# Conectar os containers na rede
oot@webapp:/#  docker network connect bigdata-net <containerA>
oot@webapp:/#  docker network connect bigdata-net <containerB>
```

Port Mapping: Expondo Serviços

-p Mapeamento Explícito

```
docker run -p 8080:80 nginx
```

Sintaxe: HOST_PORT : CONTAINER_PORT

Você define exatamente qual porta do host será usada. Ideal para produção e serviços conhecidos.

-P Mapeamento Automático

```
docker run -P nginx
```

Sintaxe: (sem argumentos)

O Docker mapeia todas as portas expostas (EXPOSE) para portas altas aleatórias no host. Útil para testes.

Prática: Port Mapping

Servidor Web (Nginx)

```
docker run -d -p 8080:80 nginx
```

Host :8080 → Container :80

Banco de Dados (Postgres)

```
docker run -d -p 5433:5432 postgres
```

Host :5433 → Container :5432

Cache (Redis)

```
docker run -d -p 6379:6379 redis
```

Host :6379 → Container :6379

Dockerfile: A Receita

Dockerfile

```
FROM node:18-alpine

WORKDIR /app

COPY . .

RUN npm install

CMD ["node", "server.js"]
```

FROM — **Imagem Base**

Define o ponto de partida (SO ou runtime). É sempre a primeira instrução.

WORKDIR — **Diretório de Trabalho**

Define onde os comandos serão executados dentro do container (como um `cd`).

COPY — **Adicionar Arquivos**

Copia arquivos do seu computador (host) para dentro da imagem.

RUN — **Executar no Build**

Roda comandos para instalar dependências ou configurar o ambiente.

CMD — **Comando Inicial**

O que o container deve fazer quando for iniciado (`docker run`).

Instruções Essenciais

Setup & Ambiente

FROM

Define a imagem base. Deve ser a primeira instrução.

```
FROM node:18-alpine
```

WORKDIR

Define o diretório de trabalho para comandos seguintes.

```
WORKDIR /app
```

ENV

Define variáveis de ambiente persistentes.

```
ENV NODE_ENV=production
```

Construção

COPY

Copia arquivos do host para o container.

```
COPY package.json .
```

ADD

Similar ao COPY, mas aceita URLs e extrai .tar.gz.

```
ADD https://src.com/file .
```

RUN

Executa comandos durante o build (cria nova layer).

```
RUN npm install
```

Execução

CMD

Comando padrão. Pode ser sobrescrito no `docker run`.

```
CMD ["npm", "start"]
```

ENTRYPOINT

Executável principal. Difícil de sobrescrever.

```
ENTRYPOINT ["python"]
```

EXPOSE

Documenta a porta que o container escuta.

```
EXPOSE 8080
```

Boas Práticas

Otimização de Layers

Combine RUN/COPY/ADD em comandos para reduzir camadas.

- `RUN apt-get update; RUN apt-get install -y curl` — Ruim
- `RUN apt-get update && apt-get install -y curl` — Bom

Cache Inteligente

Copie dependências antes do código fonte para aproveitar o cache.

- `COPY . .; RUN npm install` — Lento
- `COPY package.json .; RUN npm install; COPY . .` — Rápido

Imagens Leves

Prefira bases Alpine e use `.dockerignore` para excluir arquivos.

- `FROM node:18-alpine` — 50MB
- `FROM node:18` — 1GB+

Segurança

Não rode como root; crie um usuário e não inclua segredos na imagem.

- `RUN adduser -D appuser; USER appuser` — Seguro

Construindo Imagens

```
docker build -t myapp:1.0 .
```

Flag -t

Define a **Tag** (nome) da imagem. Essencial para identificar e versionar sua imagem.

Nome:Versão

Formato padrão repositorio/imagem:tag

Se omitir a tag, o Docker assume :latest .

Contexto (.)

O diretório que será enviado ao Docker Daemon. O ponto . significa "diretório atual".

Otimização de Contexto

O Docker envia **todos** os arquivos do contexto para o daemon antes do build. Use um arquivo .dockerignore para excluir pastas pesadas como node_modules ou .git e acelerar o processo.

Publicando no Docker Hub

1

Login

Autentique seu cliente Docker com sua conta do Docker Hub.

```
$ docker login
```

2

Tagging

Renomeie a imagem local para o formato do repositório remoto.

```
$ docker tag app:v1 user/app:v1
```

user: Seu usuário
app: Repositório
v1: Versão

3

Push

Envie as camadas da imagem para o registro público.

```
$ docker push user/app:v1
```

Boas Práticas no Docker Hub

Documentação (README)

- Descreva o propósito da imagem claramente.
- Liste as variáveis de ambiente (ENV) suportadas.
- Forneça exemplos de comandos `docker run`.

Tags e Versionamento

- Use Semantic Versioning (v1.0.0, v1.0, v1).
- Evite usar apenas `latest` em produção.
- Mantenha tags imutáveis quando possível.

Segurança

- Nunca inclua senhas ou chaves na imagem.
- Use o Docker Scan para achar vulnerabilidades.
- Prefira imagens oficiais ou verificadas.

Manutenção e CI/CD

- Automatize o build com GitHub Actions.
- Atualize a imagem base regularmente.
- Mantenha o tamanho da imagem pequeno (Alpine).

HANDS-ON

Hora do Código

```
user@cloud:~$AWS Net
```

Prática: Criando um Dockerfile

1 Preparar o Diretório

```
$ mkdir my-app
$ cd my-app
```

2 Criar a Aplicação

app.py

```
import os

print("Olá, Docker!")
print("Rodando no container.")
```

3 Criar o Dockerfile

Dockerfile

```
# 1. Imagem base leve (Python)
FROM python:3.9-alpine

# 2. Definir diretório de trabalho
WORKDIR /app

# 3. Copiar arquivos para imagem
COPY app.py .

# 4. Comando de execução
CMD ["python", "app.py"]
```

Atividade: Build e Execução

bash — 80x24

```
$ docker build -t my-python-app .
  Sending build context to Docker daemon 3.072kB
  Step 1/4 : FROM python:3.9-alpine
  ...
  Successfully built 7a8b9c0d1e2f
  Successfully tagged my-python-app:latest
$ docker images
  REPOSITORY TAG IMAGE ID SIZE
  my-python-app latest 7a8b9c0d1e2f 45MB
$ docker run --rm my-python-app
  Olá, Docker!
  Rodando no container.
```

1 **Build**

O comando build lê o Dockerfile e cria a imagem. A flag -t dá um nome fácil de lembrar.

2 **Verificar**

Confirme se a imagem foi criada corretamente e observe o tamanho otimizado (Alpine).

3 **Executar**

Rode o container! A flag --rm remove o container automaticamente após o fim da execução para não deixar lixo.

Atividade: Publicando

bash — 80x24

```
$ docker login
  Username: seu_usuario
  Password: **********
  Login Succeeded
$ docker tag my-python-app seu_usuario/python-app:v1
$ docker push seu_usuario/python-app:v1
  The push refers to repository [docker.io/seu_usuario/python-app]
  5f70bf18a086: Pushed
  8d3ac3489996: Pushed
  v1: digest: sha256:7a8b... size: 1234
```

seu_usuario/python-app:v1

Sua imagem agora está hospedada no Docker Hub, pronta para ser baixada em qualquer servidor do mundo.

Atenção ao Nome

O Docker só permite o push se a imagem estiver tagueada com seu **nome de usuário** do Docker Hub no início.

Resumo da Aula

Networking & DNS

Containers possuem isolamento de rede. O **DNS interno** permite comunicação entre serviços usando apenas o nome do container.

Port Mapping

Para acessar serviços externamente, precisamos mapear portas explicitamente (**-p 8080:80**), criando um túnel entre host e container.

Dockerfile

A receita da infraestrutura. Instruções como **FROM**, **RUN** e **CMD** permitem criar imagens reprodutíveis.

Docker Hub

O registro central para distribuição. O fluxo **Login -> Tag -> Push** permite compartilhar suas aplicações.

Conclusão & Próximos Passos

Habilidades Desbloqueadas

Domínio de Redes

Compreensão de Bridge, Host e comunicação via DNS interno.

Criação de Imagens

Escrita de Dockerfiles otimizados e uso de cache de build.

Distribuição Global

Publicação e versionamento de imagens no Docker Hub.

O Que Vem Por Aí?

"Rodar um container é fácil.

Mas e se precisarmos de 10 containers conectados?"

Docker Compose

Definição de múltiplos containers como código (YAML).

Orquestração simplificada para desenvolvimento.

UNISENAI

DEPARTAMENTO REGIONAL DE SÃO PAULO

www.sp.senai.br